EPIC-C — Forum: Posting, Commenting, Tagging, Search — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The third per-epic technical spec, building on the architecture spec (`docs/superpowers/specs/2026-07-14-askapeer-architecture-design.md`) and the EPIC-A/EPIC-B specs (verification, handles). Read the architecture spec's Section 4.2 (`community` schema) and Section 6 (search) first. EPIC-C is the largest surface area of the MVP by volume of member interaction, but architecturally the most conventional of the nine epics — most of its design questions are forum-software questions the anonymity model doesn't change, so this spec is more concise than EPIC-A/B where the identity boundary drove most of the content.

Source of truth: `docs/askapeer-prd-v0.1.md`, Section 6.1 (Must/Should/Could feature tables) and Section 6.2 (case discussion template, owned by EPIC-E but built on this epic's `posts` table).

Contents

---

1. Scope

In scope: posts (questions and, structurally, case discussions — see Section 8), threaded comments/replies, the hybrid tag taxonomy, and full-text search across posts/comments/tags.

Out of scope for this spec (owned elsewhere): - The case-discussion-specific template, de-identification checklist, and attestation — EPIC-E, which builds directly on this epic's `posts` table (a `type = case_discussion` row) rather than a separate table. - Kudos and answer ranking — EPIC-D.

This spec's threads display comments in an order EPIC-D determines (kudos-first); it doesn't compute that order itself. - Reporting a post/comment — EPIC-F, which reads `post_id`/`comment_id` as a foreign target, not owned here. - Notification triggers (new reply, mention) — EPIC-G reacts to this epic's domain events; this spec doesn't specify notification delivery.

---

2. Data model

Builds on `community.posts`, `community.comments`, `community.tags`/ `community.post_tags` already defined in the architecture spec, Section 4.2. This spec adds the **top-level category** table the hybrid taxonomy (FD-4, Option D) requires, which the architecture spec named but didn't fully model:

```
community.categories                  -- fixed, admin-managed,
small set
  id           uuid PK
  name         text unique            -- e.g. "Shoulder", "Knee",
"Research", "Career"
  description  text
  sort_order   int

community.posts
  id            uuid PK
  handle_id     uuid FK -> community.handles
  category_id   uuid FK -> community.categories        --
required; the "stable top-level
                                                          --
structure" FD-4's recommendation
                                                          -- calls
for
  type          enum(question, case_discussion)
  title         text
  body          text
  status        enum(published, removed)                -- see
Section 6
  tsv           tsvector generated                       -- see
Section 4
  created_at    timestamptz
  edited_at     timestamptz nullable

community.comments
  id                uuid PK
  post_id           uuid FK -> community.posts
  handle_id         uuid FK -> community.handles
  parent_comment_id  uuid FK -> community.comments nullable   --
threading
  body              text
  status            enum(published, removed)
  tsv               tsvector generated
  created_at        timestamptz
  edited_at         timestamptz nullable
```

```
community.tags        (id, name unique)
community.post_tags   (post_id FK, tag_id FK, primary key(post_id,
tag_id))
```

`status = removed` (rather than a hard delete) is what Section 6 needs to distinguish "content that was here and got moderated" from "content that never existed" — a hard delete would break comment threads that reply to it and would be indistinguishable from the moderation action EPIC-F performs (`remove_content`), which needs something to act on.

---

3. Tagging and the hybrid taxonomy (FD-4)

Per the architecture spec's assumption (Section 1) and the PRD's own recommendation (Section 15, FD-4, Option D): a small, fixed set of top-level `categories` (body-area plus a few professional topics — Research, Career, Equipment) chosen by the post author at creation time, plus free `tags` within that category for flexibility and search depth. Every post has exactly one category and zero-or-more tags.

This spec does not invent the category list or tag vocabulary — Andrew Renshaw has an existing body-area list per the PRD (Section 15, FD-4 discussion), and a keyword/tag vocabulary already exists for the research feed (`prototypes/research-feed/data/taxonomy.json`), but that prototype's own README is explicit that its taxonomy is "a starting point… not the final FD-4 forum taxonomy" and flags "taxonomy unification with the forum" as an open item. Reconciling the forum's category/tag list, the research feed's taxonomy, and Andrew's body-area list into one controlled vocabulary is real work that hasn't happened yet — flagged in Section 11, not assumed resolved here.

`community.categories` is deliberately admin-managed (not member-created) — a hybrid taxonomy where either half were freely extensible would collapse into pure tagging (Option B), which is the option FD-4 explicitly recommends against for MVP.

---

4. Search

Per the architecture spec, Section 6: PostgreSQL full-text search via a generated `tsvector` column (`tsv`, combining title/body for posts, body for comments) with a GIN index — no dedicated search service for MVP. Tag names and category names are included in the query construction (searching "ACL tear" should surface posts tagged `Anterior cruciate ligament` even if that exact phrase isn't in the body), not merely a `tsvector`-against-body match.

```
GET /v1/search?q=...&category=&tag=&cursor=...
  -> cursor-paginated results across posts (and, transitively,
their comments,
     surfaced as "N replies match" rather than as separate top-
level results)
  -> ranked by PostgreSQL's ts_rank combined with recency; not
kudos-weighted
     (kudos ranks answers within a thread, per EPIC-D — it doesn't
```

```
rank search
     relevance across threads, a different question)
```

The PRD (Section 6.1) marks Search itself "(to be discussed/confirmed)" even though it's in the Must-have table — an odd hedge worth surfacing rather than silently treating as fully settled (Section 11).

---

5. API endpoints

Consistent with the architecture spec's Section 5.3 principles (versioned `/v1`, cursor pagination, no `identity`-schema leakage — trivially satisfied here since this epic never touches `identity` at all).

```
POST   /v1/posts                          { category_id, type,
title, body, tag_ids[] }
GET    /v1/posts?category=&tag=&cursor=
GET    /v1/posts/:post_id
PATCH  /v1/posts/:post_id                 -- see Section 6
DELETE /v1/posts/:post_id                 -- see Section 6 (soft:
status = removed)

POST   /v1/posts/:post_id/comments        { body,
parent_comment_id? }
PATCH  /v1/comments/:comment_id
DELETE /v1/comments/:comment_id

GET    /v1/categories
GET    /v1/tags?prefix=                   -- typeahead when
composing a post
```

All write endpoints require a handle-scoped session token at `active` status (architecture spec, Section 5.2); a `suspended`/`expelled` handle's token fails on refresh as already specified there, so this epic needs no bespoke status check beyond what already exists.

---

6. Edit/delete policy

Not specified in the PRD — this spec proposes a policy and flags it for confirmation rather than assuming it:

- **Comments/posts are editable by their author within a short window** (proposed: 15 minutes) with no visible "edited" marker within that window (typo correction), and editable indefinitely after that but with an `edited_at` timestamp shown — balancing genuine correction against the integrity of a thread other members have already responded to or awarded kudos against.
- **Author self-delete is a soft delete** (`status = removed`) — same mechanism as moderator removal (EPIC-F), so a deleted post's replies aren't orphaned, consistent with Section 2's reasoning.
- **A post that already has kudos-bearing comments, or an attested case discussion (EPIC-E), should not be freely author-deletable** — deleting a case discussion after attestation would undermine the point of

the attestation existing (the PRD Section 10.3 attestation is “recorded with timestamp and linked to the member’s verified identity” specifically so it persists as a record). This spec proposes case discussions become author-delete-restricted (moderator-only) once published; ordinary questions remain author-deletable. Flagged in Section 11 since it’s this spec’s judgment call, not a PRD requirement.

---

7. Could-have features (Section 6.1)

The PRD lists three Could-have (MVP stretch) items. Specified here at a level sufficient to build if prioritised, without over-engineering for features that may not ship in MVP:

- **Best answer marker**: a nullable `community.posts.accepted_comment_id` set by the post’s own author (not moderators, not kudos-driven). Displayed above the kudos-ranked list, not replacing it — the PRD’s Must-have kudos ranking and this Could-have marker are complementary, not alternatives.
- **Image attachments**: stored in S3 per the architecture spec, Section 3, with EXIF stripping by a worker before persistence (already specified generically there) and an upload-time content-warning prompt (author-supplied, e.g. “clinical image”). A `community.attachments (id, post_id, s3_key, content_warning boolean)` table would own this; not built out further here since it’s Could-have and the architecture spec already covers the EXIF-stripping mechanism it depends on.
- **Polls**: a `community.polls (post_id, question, options jsonb)` plus a votes table — a lightweight, self-contained addition if built; no interaction with any other epic’s data.

None of these three should be assumed in the MVP build plan — flagged in Section 11 for a scope confirmation.

---

8. Personalised feed (Should-have)

PRD Section 6.1: “Home view based on tags and handles followed, with a trending/top view as fallback.” This depends entirely on `community.follows` — the unified handle-and-tag follow mechanism EPIC-B’s spec now owns (its Section 8, generalised 2026-07-14 from an earlier handle-only design; see `docs/2026-07-14-technical-specs-open-questions.md`, Section 2, for that history). This epic doesn’t own or duplicate that table — it’s a read-only consumer:

```
GET /v1/feed?cursor=...
  -> posts whose category/tags match the caller's tag-follows
(community.follows,
     target_type = tag), or whose author matches the caller's
handle-follows
     (target_type = handle), ranked by recency (not kudos — kudos
ranks answers
     within a thread, per EPIC-D, a different question from
ranking a feed of
     distinct posts)
  -> falls back to a "trending/top" view (e.g. most-kudos posts in
```

```
the last N
     days, platform-wide) when the caller follows nothing yet or
the followed-
     content result set is thin — per the PRD's own "fallback"
language
```

No new schema is introduced here — `community.follows` (EPIC-B) and `community.posts`/`post_tags` (this epic, Section 2) are sufficient to build this query. "Trending" itself isn't otherwise defined by the PRD (a simple kudos-in-a-time-window heuristic is this spec's own proposal, flagged in Section 12).

---

9. Boundaries with other epics

- **EPIC-B (follows)** owns `community.follows`; this epic reads it (Section 8) filtering both `target_type = tag` and `target_type = handle` for the personalised feed — a read-only consumer, not a second copy of the relationship.
- **EPIC-E (case discussions)** writes `type = case_discussion` posts through this epic's own `POST /v1/posts` path, but only after its own de-identification checklist and attestation gate — EPIC-E's spec should treat this epic's endpoint as the underlying mechanism it wraps, not duplicate it.
- **EPIC-D (kudos)** determines comment *display order* within a thread; this epic's `GET /v1/posts/:post_id` response includes comments in whatever order EPIC-D's ranking specifies, treating it as an injected ordering rather than this epic's own `created_at` ordering.
- **EPIC-F (moderation)** is the only actor that can set `status = removed` on someone else's content, and is what `community.moderation_actions` (architecture spec, Section 4.2) already logs.
- **EPIC-G (notifications)** subscribes to this epic's domain events (new comment, @mention parsed from `body`) — this spec doesn't define the notification payloads, only that the events exist.

---

10. Non-functional notes specific to EPIC-C

- **Rate limiting on posting/commenting**: not called out specifically in the architecture spec's Section 5.3 (which names auth and reporting endpoints) — worth extending that Redis-backed rate limiting to posting endpoints too, since a forum is an obvious spam target once real users exist.
- **@mention parsing** must resolve to a `handle_id`, never leak whether a mentioned handle exists if it's `expelled` (mentioning an expelled handle should behave identically to mentioning a nonexistent one, not reveal status).
- `tsv` **generated columns** on both `posts` and `comments` keep search index maintenance in the database rather than the application layer, consistent with the architecture spec's general preference for schema-level guarantees over code discipline (Section 4).

---

11. Test plan

- **Category/tag integrity**: a post always has exactly one `category_id` (not nullable); zero-or-more tags.
- **Soft delete**: deleting a post sets `status = removed`, doesn't cascade-delete comments; a `removed` post's comments remain readable in-thread but the post body is replaced with a removal notice (or however EPIC-F's spec eventually defines display — flagged for reconciliation there).
- **Search ranking**: a query matching a tag name surfaces the tagged post even when the exact phrase isn't in the body (Section 4).
- **Edit window**: a comment edited within the proposed 15-minute window shows no `edited_at` marker; after, it does.
- **Case-discussion delete restriction**: an attested case discussion (once EPIC-E exists to create one) cannot be author-deleted, only moderator-removed.
- **Personalised feed composition**: a post tagged with a followed tag, or authored by a followed handle, appears in the feed (Section 8); a post matching neither doesn't; the trending fallback activates when the caller follows nothing.

---

12. Open questions

- **Taxonomy unification** (Section 3): the forum's `category`/`tag` vocabulary, Andrew Renshaw's body-area list, and the research feed's `taxonomy.json` are three overlapping-but-not-identical vocabularies right now. Needs a single controlled vocabulary decided before build, and FD-4 needs formal closure regardless (it's still an open stakeholder decision, not just an implementation detail).
- **Search's "(to be discussed/confirmed)" hedge** (Section 4): the PRD lists full-text search as Must-have but flags it for discussion in the same breath — worth Adrian confirming with Paul/Andrew whether that hedge is still live or was resolved verbally and just never updated in the document.
- **Edit/delete policy** (Section 6): entirely this spec's proposal, not a PRD requirement — the 15-minute window, the case-discussion delete restriction, and whether comments with kudos already awarded should be edit-restricted too, all need sign-off.
- **Could-have scope confirmation** (Section 7): whether any of best-answer marker, image attachments, or polls are actually being built for MVP launch, or deferred — affects sequencing/resourcing, not architecture.
- **"Trending" definition** (Section 8): a kudos-in-a-time-window heuristic is this spec's own placeholder for the PRD's unspecified fallback view — needs a concrete definition (window length, whether it's platform-wide or category-scoped) before build.